

Министерство образования и науки Российской Федерации
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Санкт-Петербургский государственный университет промышленных
технологий и дизайна»

Институт информационных технологий и автоматизации

Кафедра: Интеллектуальных систем и защиты информации
Направление подготовки: 09.03.01 Информационные системы и технологии
Профиль подготовки: Информационные системы и сетевые технологии

ОТЧЕТ

по производственной практике

Тема: «Проектирование микросервисной архитектуры SaaS-платформы
CampusForge»

Преподаватель: Бусыгин К. Н.

Обучающийся: Беляков Д.Н.

Курс: 3 Учебная группа: 3-МД-16

Санкт-Петербург

2026

Содержание

1. Постановка задачи
 - 1.1. Общее описание разрабатываемого решения
 - 1.2. Актуальность решения
 - 1.3. Целевая аудитория
 - 1.4. Цели и задачи проектирования
 - 1.5. Границы первой версии продукта
2. Архитектура разрабатываемого решения
 - 2.1. Общий архитектурный подход
 - 2.2. Состав микросервисов
 - 2.3. Взаимодействие сервисов
 - 2.4. Данные и владение таблицами
 - 2.5. Клиентская часть
 - 2.6. Событийная модель и realtime-функции
 - 2.7. Запуск системы через Docker Compose
3. стек используемых технологий
 - 3.1. TypeScript и JavaScript
 - 3.2. React, Vite и Tailwind CSS
 - 3.3. NestJS и backend-сервисы
 - 3.4. PostgreSQL и Prisma ORM
 - 3.5. Redis, Kafka и MinIO
 - 3.6. Docker Compose и Nginx
4. Функциональная модель решения
5. Модель данных

6. Организация разработки

7. Ожидаемые результаты

Заключение

1. Постановка задачи

1.1. Общее описание разрабатываемого решения

В рамках работы проектируется web-платформа CampusForge, предназначенная для цифровизации проектной деятельности университета. Система должна объединять студентов, преподавателей, кафедры, университеты, компании-партнеры, менторов и экспертов в едином цифровом пространстве. Основная задача решения заключается в создании платформы, которая позволит управлять проектами, формировать команды, сопровождать индивидуальные и командные работы, фиксировать результаты, организовывать оценивание и формировать проектное портфолио пользователей.

CampusForge рассматривается не как отдельная доска задач и не как простой каталог проектов, а как комплексная SaaS-система, объединяющая несколько процессов: регистрацию пользователей, создание профилей, подключение организаций, публикацию проектов, прием заявок, приглашение участников, ведение задач, загрузку материалов, оценивание, уведомления, аналитику и управление подписками.

Решение сразу проектируется как микросервисная система. Это означает, что основные предметные области выделяются в самостоятельные backend-сервисы с собственными зонами ответственности, API-контрактами, моделями данных и механизмами взаимодействия. Такой подход позволяет развивать платформу по частям, масштабировать отдельные сервисы и уменьшать связанность между доменами системы.

Клиентская часть системы предоставляет пользователям веб-интерфейс для работы с профилем, проектами, задачами, организациями, уведомлениями и административными разделами. Серверная часть состоит из набора микросервисов, которые взаимодействуют через API Gateway, HTTP/gRPC-запросы и события Kafka. Вся система должна запускаться через

Docker Compose, что обеспечивает воспроизводимое окружение разработки и проверки.

Ключевой результат проектирования — описание и подготовка архитектуры, которая поддерживает полный жизненный цикл проектной деятельности: от появления идеи и поиска команды до ведения задач, оценки результата, архивации проекта и отображения работы в портфолио студента.

1.2. Актуальность решения

Проектная деятельность является важной частью современного образовательного процесса. В IT, инженерии, аналитике, дизайне и предпринимательстве ценность обучения всё чаще определяется не только набором изученных дисциплин, но и реальными результатами: созданными приложениями, исследовательскими проектами, кейсами компаний, командной работой и участием в хакатонах.

Во многих образовательных организациях такие процессы ведутся фрагментарно. Обсуждения находятся в мессенджерах, задачи фиксируются в таблицах, файлы хранятся в облачных папках, заявки собираются через формы, а итоговая отчетность формируется вручную. Из-за этого теряется прозрачность: сложно понять, кто участвует в проекте, какой вклад внес каждый студент, какие дедлайны нарушены, какие проекты завершены и какие компетенции были проявлены.

Для студентов актуальность решения связана с возможностью находить команды, участвовать в проектах, демонстрировать реальные навыки и собирать портфолио. Для преподавателей платформа полезна как инструмент сопровождения работ, проверки прогресса, комментирования и оценивания. Для университетов она становится способом систематизировать проектную активность и получать аналитику. Для компаний CampusForge может быть каналом поиска молодых специалистов через реальные проектные результаты.

Таким образом, разрабатываемая система решает практическую задачу организации проектной деятельности в цифровой форме. Она позволяет заменить разрозненные инструменты единой платформой, где есть роли, статусы, задачи, документы, уведомления, оценки и отчеты.

1.3. Целевая аудитория

Группа пользователей	Потребности	Возможности в CampusForge
Студенты	Поиск команды, участие в проектах, развитие портфолио, демонстрация навыков	Профиль, навыки, каталог проектов, заявки, задачи, документы, отзывы
Преподаватели	Сопровождение индивидуальных и командных работ, контроль прогресса, оценивание	Кабинет преподавателя, комментарии, критерии, оценки, аналитика
Университеты и кафедры	Управление проектной деятельностью, отчетность, работа с группами	Workspace, учебные группы, официальные проекты, роли, отчеты
Компании-партнеры	Публикация кейсов, поиск студентов, проведение мероприятий	Company Hub, кейсы, стажировки, эксперты, оценивание
Менторы и жюри	Помощь командам и оценка результатов	Роли в проектах, комментарии, отзывы, баллы
Администраторы платформы	Управление системой, модерация, контроль пользователей и организаций	Admin-панель, модерация, управление тарифами и лимитами

Целевая аудитория разделяется не только по типам пользователей, но и по уровню доступа. Один и тот же человек может быть студентом в одном проекте, капитаном команды в другом, участником организации и автором личного проекта в открытой зоне платформы. Поэтому система должна поддерживать глобальные роли, роли внутри организации и роли внутри проекта.

1.4. Цели и задачи проектирования

Целью проектирования является создание архитектуры и функциональной модели web-платформы CampusForge, которая позволяет управлять проектной деятельностью университета и взаимодействием участников проектного процесса.

Для достижения цели необходимо решить следующие задачи:

- сформировать общую модель платформы и определить основные пользовательские сценарии;
- спроектировать микросервисную backend-архитектуру с разделением ответственности между сервисами;
- определить состав внешнего REST API и внутренних сервисных контрактов;
- описать модель данных и распределить таблицы по микросервисам;
- подобрать технологический стек для frontend, backend, базы данных, кэша, событийной шины и файлового хранилища;
- описать механизм авторизации и разграничения доступа;
- определить подход к запуску всей системы через Docker Compose;
- выделить функции первой версии продукта и направления дальнейшего развития.

1.5. Границы первой версии продукта

Первая версия CampusForge должна включать функции, необходимые для запуска базового жизненного цикла проектной деятельности. В нее входят регистрация и вход пользователей, профили, организации, проекты, участники, заявки, приглашения, задачи, документы, оценивание, уведомления и базовые административные функции.

В первой версии не требуется реализовывать все возможные расширения платформы. Например, расширенный AI-подбор участников,

мобильное приложение, полноценная интеграция с LMS и онлайн-платежи могут быть вынесены в следующие этапы. При этом архитектура должна сразу учитывать возможность подключения таких функций без полного изменения структуры backend.

Приоритет	Функция первой версии
1	Регистрация, авторизация, JWT access/refresh tokens, подтверждение email
1	Профиль пользователя, навыки, ссылки и документы
1	Организации, рабочие пространства, учебные группы и участники
1	Создание проектов разных типов и форматов
1	Заявки и приглашения в проектные команды
1	Kanban-доска, задачи, комментарии и вложения
1	Критерии оценивания, оценки и отзывы
1	Уведомления о ключевых событиях
2	Подписки, лимиты и счетчики использования
2	Поиск и базовая аналитика
3	Расширенные хакатоны, сертификаты, AI-рекомендации, real-time чат

2. Архитектура разрабатываемого решения

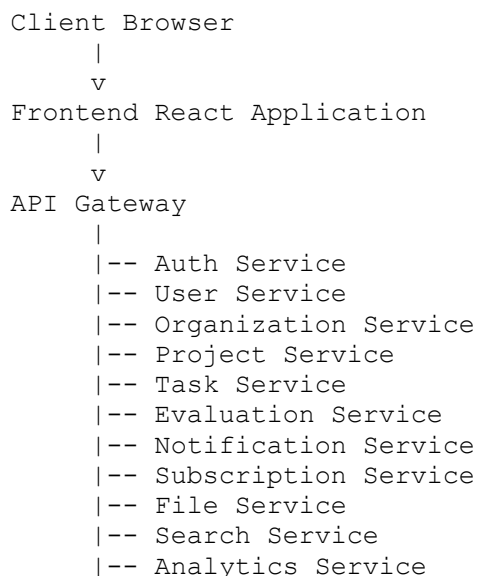
2.1. Общий архитектурный подход

CampusForge проектируется и реализуется как микросервисная web-платформа. Основные домены системы выделяются в самостоятельные сервисы: аутентификация, пользователи, организации, проекты, задачи, оценивание, уведомления, подписки, файлы, поиск, аналитика, модерация и администрирование. Каждый сервис имеет собственную область ответственности и предоставляет API для выполнения операций в своей предметной области.

Внешняя точка входа в backend — API Gateway. Клиентское приложение не обращается напрямую к отдельным сервисам. Все запросы проходят через Gateway, который отвечает за маршрутизацию, проверку токена, передачу контекста пользователя, rate limiting, логирование и единый формат ответов. Такой подход упрощает frontend, скрывает внутреннюю структуру backend и позволяет менять внутренние сервисы без изменения публичного API.

Взаимодействие между сервисами строится на двух механизмах. Для операций, где требуется немедленный ответ, используются синхронные HTTP или gRPC-запросы. Для действий, которые должны выполняться асинхронно, используется Kafka. Например, Project Service может создать проект и опубликовать событие project.created, после чего Notification Service создаст уведомление, Search Service обновит индекс, а Analytics Service обновит статистику.

Микросервисный подход выбран как основной, потому что CampusForge содержит несколько крупных доменов, которые могут развиваться и масштабироваться независимо. Уведомления, файлы, поиск, аналитика и подписки имеют разную нагрузку и разные требования к данным. Выделение этих частей в отдельные сервисы упрощает поддержку и делает систему более гибкой.



```

|-- Moderation Service
|-- Admin Service
|
v
PostgreSQL + Redis + Kafka + MinIO

```

2.2. Состав микросервисов

Ниже приведен состав основных микросервисов CampusForge и их назначение. Разделение выполнено по бизнес-доменам, чтобы каждый сервис решал ограниченный набор задач и не смешивал ответственность с другими частями платформы.

Сервис	Назначение
API Gateway	Единая точка входа, маршрутизация, проверка JWT, передача user context
Auth Service	Регистрация, вход, refresh-токены, подтверждение email, восстановление пароля
User Service	Профили пользователей, навыки, ссылки, документы, студенческие и преподавательские данные
Organization Service	Университеты, кафедры, факультеты, компании, workspace, участники, учебные группы
Project Service	Проекты, участники, заявки, приглашения, проектные роли, ссылки и документы
Task Service	Kanban-доски, колонки, задачи, исполнители, комментарии и вложения
Evaluation Service	Критерии, оценки проектов, баллы, отзывы преподавателей, менторов и экспертов
Notification Service	Уведомления, настройки доставки, история отправки
Subscription Service	Тарифы, подписки, лимиты, счетчики использования, счета и платежи
File Service	Загрузка, хранение, выдача и удаление файлов через MinIO
Search Service	Поиск по пользователям, организациям, проектам и навыкам
Analytics Service	Отчеты, статистика, метрики активности и проектной деятельности

Moderation Service	Жалобы, проверка организаций, блокировки, скрытие контента
Admin Service	Системные административные операции и единый интерфейс управления

Каждый сервис может иметь отдельный NestJS-проект внутри монопогера. Общие типы, DTO, события, guards, конфигурация, логирование и вспомогательные библиотеки выносятся в директорию `libs`. Это позволяет избежать дублирования и при этом сохранить изоляцию доменной логики.

2.3. Взаимодействие сервисов

Синхронное взаимодействие используется в случаях, когда сервису требуется получить данные до завершения операции. Например, Project Service может обратиться к Organization Service для проверки существования организации, а Task Service — к Project Service для проверки участия пользователя в проекте.

Асинхронное взаимодействие используется для событий, которые не должны блокировать основную бизнес-операцию. Например, при создании заявки в проект основная операция должна быстро сохранить заявку, а уведомление, аналитика и обновление поискового индекса могут быть выполнены после публикации события.

```

Project Service
  creates project application
    |
    v
Kafka topic: project-events
    |
    |-- Notification Service creates notification
    |-- Analytics Service updates metrics
    |-- Search Service updates indexed data

```

Стандарт события содержит идентификатор события, тип события, источник, дату возникновения и полезную нагрузку. Это упрощает трассировку, повторную обработку и интеграцию новых потребителей событий.

```

{
  "eventId": "fcb2c674-dc85-4061-a9c7-4f4a73abf61e",

```

```

    "eventType": "project.application_created",
    "source": "project-service",
    "occurredAt": "2026-06-22T16:40:00Z",
    "payload": {
      "projectId": "70f01b1c-d624-4c61-a5f3-87364e91d5e1",
      "applicantUserId": "1bb8eec1-2ed5-40a3-a4cf-9e30ecdf2d50"
    }
  }
}

```

2.4. Данные и владение таблицами

В микросервисной архитектуре каждый сервис владеет своими данными. Это означает, что только сервис-владелец может создавать, изменять и удалять записи в своих таблицах. Другие сервисы используют API или события. Такой подход предотвращает сильную связанность и делает границы доменов понятными.

При использовании отдельных баз данных между сервисами не создаются жесткие foreign key. Например, Task Service может хранить project_id и assignee_user_id, но не должен иметь прямой внешний ключ на таблицы Project Service и User Service. Проверка существования проекта или пользователя выполняется через соответствующий сервис или через read-модель.

Микросервис	Таблицы
Auth Service	users, refresh_tokens, email_verification_tokens, password_reset_tokens
User Service	user_profiles, student_profiles, teacher_profiles, skills, user_skills, user_links, user_documents
Organization Service	organizations, organization_domains, organization_requests, organization_workspaces, organization_links, organization_contacts, academic_groups, organization_memberships, organization_member_roles, user_invitations
Access Control	roles, permissions, role_permissions
Project Service	projects, project_members, project_member_roles, project_applications,

	project_invitations, project_skills, project_links, project_documents
Task Service	task_boards, task_columns, tasks, task_assignees, task_comments, task_attachments
Evaluation Service	evaluation_criteria_sets, evaluation_criteria, project_evaluations, project_evaluation_scores, project_reviews
Notification Service	notifications, notification_settings, notification_deliveries
Subscription Service	subscription_plans, subscription_plan_limits, subscriptions, usage_counters, invoices, payments
File Service	files или физическое хранение файлов и выдача storage_key
Search Service	search_index, search_logs или внешний поисковый индекс
Analytics Service	analytics_events, daily_project_stats, organization_metrics, project_metrics
Moderation Service	moderation_reports, moderation_actions
Admin Service	admin_audit_logs, admin_actions

На уровне инфраструктуры может использоваться один экземпляр PostgreSQL, разделенный на логические базы или схемы для сервисов. При этом архитектурное правило сохраняется: сервис работает только со своей областью данных. Такой подход удобен для разработки и одновременно соответствует микросервисному разделению.

2.5. Клиентская часть

Клиентская часть реализуется как SPA-приложение на React и TypeScript. Frontend взаимодействует только с API Gateway и не знает о внутренней структуре backend. Это позволяет добавлять, изменять и масштабировать backend-сервисы без изменения большого количества клиентского кода.

Основные страницы включают регистрацию, вход, профиль пользователя, каталог проектов, страницу проекта, доску задач, список организаций, кабинет организации, уведомления, оценивание и

административные разделы. Интерфейс должен быть адаптивным и удобным для работы на ноутбуках, настольных компьютерах и планшетах.

Для организации frontend-кода используется Feature-Sliced Design. Такой подход помогает разделить приложение на уровни app, pages, widgets, features, entities и shared. Он хорошо подходит для крупных интерфейсов, где есть много независимых пользовательских сценариев.

```
src/  
  app/  
  pages/  
  widgets/  
  features/  
  entities/  
  shared/
```

2.6. Событийная модель и realtime-функции

Kafka используется как событийная шина между сервисами. Сервисы публикуют события о произошедших изменениях, а заинтересованные сервисы обрабатывают их независимо. Это снижает связанность между сервисами и позволяет добавлять новые обработчики без изменения основного бизнес-кода.

Например, событие task.assigned может быть обработано Notification Service для создания уведомления, Analytics Service для обновления статистики и, при необходимости, Search Service для обновления индекса. При этом Task Service не должен знать, какие именно действия выполняют другие сервисы после назначения задачи.

Realtime-функции реализуются через WebSocket. Они нужны для мгновенной доставки уведомлений, обновления статусов задач, отображения новых комментариев и будущего командного чата. WebSocket Gateway может быть отдельным сервисом или частью Notification Service, но в любом случае он должен получать события из Kafka и отправлять их подключенным клиентам.

Событие	Источник	Потребители
user.registered	Auth Service	User Service, Analytics Service, Notification Service
project.created	Project Service	Search Service, Analytics Service, Notification Service
project.application_created	Project Service	Notification Service, Analytics Service
task.assigned	Task Service	Notification Service, Analytics Service
evaluation.submitted	Evaluation Service	Notification Service, Analytics Service
subscription.limit_exceeded	Subscription Service	Notification Service, Admin Service

2.7. Запуск системы через Docker Compose

Проект полностью запускается через Docker Compose. Это означает, что все основные компоненты системы описываются в `docker-compose.yml` и поднимаются одной командой. Такой подход позволяет запускать одинаковое окружение на разных машинах без ручной установки PostgreSQL, Redis, Kafka, MinIO, Nginx и отдельных backend-сервисов.

```
docker compose up --build
```

В состав Docker Compose входят frontend-приложение, API Gateway, backend-микросервисы, PostgreSQL, Redis, Kafka, MinIO и Nginx. Дополнительно могут быть подключены Kafka UI, pgAdmin или MinIO Console для удобства разработки и администрирования.

Контейнер	Назначение
frontend	Клиентское приложение на React
api-gateway	Единая точка входа для REST API
auth-service	Аутентификация и учетные записи
user-service	Профили и пользовательские данные
organization-service	Организации и рабочие пространства
project-service	Проекты и команды
task-service	Задачи и доски
notification-service	Уведомления и WebSocket
postgres	Основная реляционная база данных

redis	Кэш и временные данные
kafka	Событийная шина
minio	Объектное хранилище файлов
nginx	Reverse proxy

3. Стек используемых технологий

3.1. TypeScript и JavaScript

В качестве основного языка разработки используется TypeScript. Он применяется как на frontend, так и на backend. TypeScript расширяет JavaScript статической типизацией, что особенно важно для крупной системы с большим количеством DTO, API-контрактов, событий, ролей и моделей данных.

Использование одного языка в клиентской и серверной частях упрощает разработку общих типов, контрактов событий и интерфейсов. Например, структуры request/response DTO и форматы событий Kafka можно хранить в общей библиотеке contracts и использовать в разных сервисах.

Для CampusForge это особенно важно, потому что система содержит много взаимосвязанных сущностей: пользователей, профили, организации, проекты, участников, задачи, оценки, уведомления и подписки. Ошибка в типах данных между сервисами может привести к сложным проблемам, поэтому статическая типизация помогает уменьшить количество таких ошибок.

3.2. React, Vite и Tailwind CSS

React используется для разработки клиентского интерфейса. Он хорошо подходит для построения сложных SPA-приложений, где интерфейс состоит из множества переиспользуемых компонентов: карточек проектов, списков участников, досок задач, форм заявок, таблиц оценивания и административных экранов.

Vite используется как инструмент сборки frontend-приложения. Он обеспечивает быстрый запуск dev-сервера, удобную разработку и современную сборку проекта. Для приложения с большим количеством страниц и компонентов скорость локальной разработки имеет большое значение.

Tailwind CSS используется для стилизации интерфейса. Он позволяет быстро создавать адаптивные и единообразные интерфейсы без написания большого количества отдельных CSS-файлов. Для CampusForge это удобно, потому что платформа включает множество экранов: профиль, каталог проектов, карточку проекта, Kanban-доску, кабинет организации, уведомления и админ-панель.

Feature-Sliced Design используется как методология организации frontend-кода. Она позволяет разделить страницы, виджеты, фичи, сущности и общие компоненты. Благодаря этому интерфейс не превращается в набор хаотичных компонентов и остается удобным для развития.

3.3. NestJS и backend-сервисы

Backend реализуется на NestJS. Каждый микросервис представляет собой отдельное NestJS-приложение или отдельный сервис внутри monorepo. NestJS подходит для такой архитектуры, потому что изначально поддерживает модульность, dependency injection, guards, pipes, interceptors, decorators и интеграции с транспортом микросервисов.

API Gateway также реализуется на NestJS. Он принимает внешние REST-запросы, проверяет JWT, формирует контекст пользователя и перенаправляет запросы во внутренние сервисы. Внутренние сервисы могут взаимодействовать через HTTP, gRPC или Kafka в зависимости от типа операции.

Для каждого сервиса проектируется отдельная зона ответственности. Например, Auth Service отвечает только за учетные записи и токены, Project

Service — за проекты и участников, Task Service — за задачи и комментарии, Notification Service — за уведомления. Такое разделение уменьшает связанность кода и позволяет масштабировать сервисы независимо.

REST API используется как основной внешний API. Он хорошо подходит для CRUD-операций, загрузки файлов, административных сценариев, пагинации, фильтрации и интеграции со Swagger/OpenAPI. Для realtime-функций применяется WebSocket.

3.4. PostgreSQL и Prisma ORM

PostgreSQL используется как основная реляционная СУБД. Система содержит большое количество сущностей со связями: пользователи, организации, группы, проекты, участники, заявки, задачи, оценки, подписки и платежи. Для таких данных реляционная модель подходит лучше, чем полностью документное хранилище.

PostgreSQL поддерживает транзакции, индексы, ограничения уникальности, JSONB-поля, полнотекстовый поиск и надежную работу со сложными запросами. Это делает его подходящим вариантом для платформы, где важна целостность данных и предсказуемое поведение.

Prisma ORM используется для описания схемы данных, генерации типизированного клиента и выполнения миграций. В микросервисной архитектуре каждый сервис может иметь собственный Prisma schema или собственный набор моделей. Это позволяет контролировать, какие таблицы принадлежат конкретному сервису.

Для разработки возможно использование одного PostgreSQL-инстанса с несколькими схемами или базами. При дальнейшем развертывании каждый сервис может получить собственную базу данных. Архитектурное правило остается одинаковым: сервис работает только со своими таблицами и не изменяет чужие данные напрямую.

3.5. Redis, Kafka и MinIO

Redis используется для кэша, временных данных, rate limiting, хранения короткоживущих состояний и ускорения часто повторяющихся операций. Например, Redis может использоваться для хранения временных кодов подтверждения, счетчиков запросов, кэша профилей или состояния WebSocket-подключений.

Kafka используется как основная событийная шина между микросервисами. Она позволяет передавать события надежно и асинхронно. Это важно для действий, которые не должны блокировать основной пользовательский запрос: отправка уведомлений, обновление поискового индекса, пересчет аналитики, проверка лимитов или запись событий активности.

MinIO используется как S3-compatible объектное хранилище. В CampusForge пользователи загружают аватары, документы, презентации, изображения, материалы проектов и вложения к задачам. Хранить такие файлы напрямую в PostgreSQL нецелесообразно, поэтому файлы размещаются в объектном хранилище, а в базе сохраняются только метаданные и storage_key.

Инструмент	Назначение в проекте
Redis	Кэш, временные данные, rate limiting, состояние realtime-подключений
Kafka	Асинхронные события между микросервисами
MinIO	Хранение файлов, документов, изображений и вложений

3.6. Docker Compose и Nginx

Docker используется для контейнеризации всех компонентов платформы. Каждый сервис запускается в собственном контейнере: frontend, API Gateway, backend-сервисы, база данных, Redis, Kafka, MinIO и Nginx. Это

делает окружение воспроизводимым и снижает зависимость от конкретной локальной машины.

Docker Compose описывает все сервисы в одном файле и позволяет запускать проект одной командой. Для CampusForge это особенно важно, потому что система состоит из нескольких микросервисов и инфраструктурных компонентов. Без контейнеризации запуск такого окружения вручную был бы сложным и нестабильным.

Nginx используется как reverse проху. Он может принимать внешние HTTP-запросы, проксировать frontend и API Gateway, настраивать CORS, TLS, сжатие и маршрутизацию. В production-сценарии Nginx также может использоваться для отдачи статических файлов frontend-приложения.

4. Функциональная модель решения

Функциональная модель CampusForge строится вокруг проектной деятельности. Центральной сущностью является проект. Вокруг проекта формируются участники, роли, задачи, документы, заявки, приглашения, оценки, уведомления и аналитика. При этом проект может принадлежать открытой зоне, университетскому workspace или компании.

Платформа поддерживает несколько типов проектов: личный проект, курсовая работа, ВКР, учебный командный проект, хакатонный проект, кейс от компании, стартап и исследовательский проект. Каждый тип может иметь собственные правила доступа, роли и сценарии оценивания.

Пользователь начинает работу с регистрации и заполнения профиля. В профиле указываются навыки, ссылки, описание, документы и история проектов. Эти данные используются для поиска участников, рекомендаций и формирования портфолио.

Организация может представлять университет, факультет, кафедру, компанию, акселератор или студенческое объединение. Организация создает

рабочее пространство, приглашает участников, формирует группы, публикует официальные проекты и получает аналитику.

Командная работа внутри проекта организуется через доску задач. Задачи имеют статус, приоритет, дедлайн, исполнителей, комментарии и вложения. Это позволяет фиксировать процесс работы, а не только итоговый результат.

Оценивание выполняется через критерии и баллы. Преподаватель, руководитель, ментор, эксперт компании или жюри могут оставлять оценки и отзывы. После завершения проекта результат сохраняется в портфолио участника.

4.1. Основные пользовательские сценарии

- Студент регистрируется, заполняет профиль, указывает навыки и подает заявку в проект.
- Автор проекта создает проект, указывает тип, описание, требуемые навыки, формат и максимальное число участников.
- Капитан или преподаватель рассматривает заявки и приглашает участников в команду.
- Команда ведет работу через Kanban-доску, задачи, комментарии и документы.
- Преподаватель или эксперт оценивает проект по критериям и оставляет отзыв.
- Завершенный проект попадает в портфолио участника и может использоваться как подтверждение опыта.
- Администратор организации анализирует активность студентов, состояние проектов и результаты оценивания.

Модуль	Основные функции
Auth	Регистрация, вход, refresh-токены, подтверждение email, восстановление пароля

Users	Профили, навыки, ссылки, документы, портфолио
Organizations	Организации, workspace, группы, участники, приглашения
Projects	Проекты, роли, заявки, приглашения, документы, ссылки
Tasks	Доски, колонки, задачи, комментарии, вложения
Evaluations	Критерии, оценки, баллы, отзывы
Notifications	Уведомления и настройки доставки
Subscriptions	Тарифы, лимиты, подписки, использование
Admin	Администрирование, модерация, управление платформой

5. Модель данных

Модель данных CampusForge отражает основные домены системы. Данные разделяются по микросервисам. Для каждого домена определяется сервис-владелец, который отвечает за создание, изменение и удаление соответствующих записей.

Auth Service хранит учетные записи и данные безопасности. User Service хранит расширенные профили и навыки. Organization Service хранит организации, рабочие пространства, группы и участников. Project Service хранит проекты и командные взаимодействия. Task Service хранит задачи и комментарии. Evaluation Service хранит критерии и оценки. Notification Service хранит уведомления. Subscription Service хранит тарифы, подписки и лимиты.

Такой подход позволяет избежать ситуации, когда один сервис напрямую меняет данные другого сервиса. Например, при удалении участника из проекта Project Service публикует событие, а Notification Service создает уведомление. Если Analytics Service должен обновить статистику, он также обрабатывает событие, но не обращается напрямую к таблицам Project Service.

Домен	Ключевые сущности
Пользователи	users, user_profiles, student_profiles, teacher_profiles, skills, user_skills
Организации	organizations, organization_workspaces, academic_groups, organization_memberships
Проекты	projects, project_members, project_applications, project_invitations, project_documents
Задачи	task_boards, task_columns, tasks, task_assignees, task_comments
Оценивание	evaluation_criteria_sets, evaluation_criteria, project_evaluations, project_reviews
Уведомления	notifications, notification_settings, notification_deliveries
Подписки	subscription_plans, subscription_plan_limits, subscriptions, usage_counters
Модерация	moderation_reports, moderation_actions

Для идентификаторов используется UUID. Для временных полей используется TIMESTAMPTZ. Для enum-полей на уровне базы могут использоваться VARCHAR с ограниченным набором значений или отдельные PostgreSQL enum-типы. В Prisma такие значения описываются через enum.

В микросервисной архитектуре особое значение имеет согласованность данных. Так как разные сервисы работают со своими таблицами, для межсервисных процессов применяются события. Например, создание пользователя может привести к созданию профиля, отправке email-уведомления и обновлению статистики регистраций.

6. Организация разработки

Разработка CampusForge начинается с подготовки монорепо, в котором находятся frontend-приложение, API Gateway, backend-микросервисы, общие библиотеки и документация. Такой подход позволяет хранить сервисы в одном репозитории, но развивать их как отдельные приложения.

```
campusforge/
  apps/
```

```
frontend/  
api-gateway/  
auth-service/  
user-service/  
organization-service/  
project-service/  
task-service/  
evaluation-service/  
notification-service/  
subscription-service/  
file-service/  
search-service/  
analytics-service/  
moderation-service/  
admin-service/  
libs/  
  common/  
  contracts/  
  config/  
  database/  
  logger/  
  auth/  
  events/  
docs/  
docker-compose.yml  
.env.example  
README.md
```

На первом этапе создается инфраструктура: Docker Compose, PostgreSQL, Redis, Kafka, MinIO, базовая конфигурация NestJS-сервисов и frontend-приложение. Затем реализуется Auth Service, потому что от него зависят остальные части системы. После этого разрабатываются User Service, Organization Service, Project Service и Task Service.

Каждый сервис должен иметь собственные контроллеры, DTO, сервисный слой, Prisma schema, переменные окружения, обработчики событий и тесты. Общие контракты событий и API-типов должны находиться в `libs/contracts`. Общие guards, decorators и утилиты авторизации — в `libs/auth`.

Запуск разработки выполняется через Docker Compose. Это позволяет одновременно поднять все сервисы и инфраструктурные компоненты. Для проверки состояния сервисов можно использовать healthcheck, логирование и отдельные UI-инструменты, например Kafka UI или pgAdmin.

```
cp .env.example .env  
docker compose up --build
```

7. Ожидаемые результаты

В результате проектирования должна быть подготовлена архитектура микросервисной платформы CampusForge, включающая frontend-приложение, API Gateway, набор backend-сервисов, базу данных, кэш, событийную шину, файловое хранилище и инфраструктуру запуска через Docker Compose.

Ожидаемый технический результат выражается в создании основы для разработки системы, где каждый домен имеет четкую зону ответственности. Это позволит последовательно реализовывать функциональность, не смешивая бизнес-логику пользователей, проектов, задач, оценивания, уведомлений и подписок.

Ожидаемый пользовательский результат заключается в создании платформы, которая упрощает организацию проектной деятельности: студенты смогут искать команды и вести проекты, преподаватели — сопровождать и оценивать работы, организации — управлять проектной активностью, компании — публиковать кейсы и находить участников по реальным результатам.

7.1. Возможные направления развития

- интеграция с GitHub для отображения репозиторий и активности;
- интеграция с LMS университета;
- реализация полноценного real-time чата команд;
- расширенные хакатоны, лидерборды и сертификаты;
- AI-подбор участников и проектов по навыкам;
- публичная витрина лучших студенческих проектов;
- мобильное приложение;
- расширенная аналитика для университетов и компаний;
- онлайн-платежи и интеграция с платежными провайдерами.

Заключение

CampusForge представляет собой микросервисную SaaS-платформу для цифровизации проектной деятельности университета. Система объединяет пользователей, организации, проекты, команды, задачи, документы, оценки, уведомления, подписки и аналитику в единой цифровой среде.

Выбранная архитектура предполагает немедленное проектирование решения как набора независимых микросервисов. Такой подход соответствует масштабу системы, так как разные домены имеют разные требования к данным, нагрузке и развитию. API Gateway обеспечивает единую точку входа, Kafka связывает сервисы через события, PostgreSQL хранит структурированные данные, Redis ускоряет работу с временными данными, MinIO отвечает за файлы, а Docker Compose обеспечивает запуск всей системы.

Разработанное решение позволяет перейти от разрозненных чатов, таблиц и файлов к управляемому процессу проектной деятельности. Для студентов это означает удобное портфолио и поиск команд, для преподавателей — сопровождение и оценивание работ, для университетов — отчетность и аналитику, для компаний — доступ к студентам через реальные проектные результаты.